

CI Pipeline Definition

Powerweave Smart Task Escalation Engine — GitHub Actions

Project: powerweave-escalation **Platform:** GitHub Actions **Date:** 26 May 2026 **Version:** 1.0

4
Jobs

2
Triggers

Node
22
Runtime

80%
Coverage
Gate

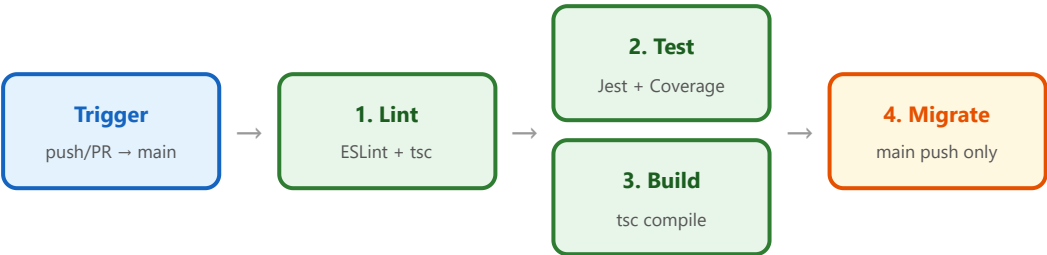
ubuntu
Runner

main
Protected
Branch

1. OVERVIEW

The CI pipeline runs automatically on every push to `main` and on every pull request targeting `main`. It enforces code quality, correctness, and safe database migrations through four sequential/parallel jobs.

Pipeline Flow



2. TRIGGERS

Event	Branch Filter	Jobs Executed
push	main	Lint → Test + Build (parallel) → Migrate
pull_request	targeting main	Lint → Test + Build (parallel) (migrate skipped)

3. JOB DEFINITIONS

1 Lint & Type Check

Property	Value
Runner	ubuntu-latest

Depends on	— (runs first)
Blocks	Test, Build jobs

#	Step	Command / Action	Purpose
1	Checkout	<code>actions/checkout@v4</code>	Clone repository at commit SHA
2	Setup Node	<code>actions/setup-node@v4</code> (v22, npm cache)	Install Node 22 with npm dependency caching
3	Install deps	<code>npm ci</code>	Clean install from package-lock.json
4	ESLint	<code>npm run lint</code>	Enforce code style across src/ and tests/
5	Type check	<code>npx tsc --noEmit</code>	Verify TypeScript compiles without errors (no output)

Failure condition: Any ESLint error or TypeScript compiler error fails this job and blocks all downstream jobs.

2 Unit Tests & Coverage

Property	Value
Runner	<code>ubuntu-latest</code>
Depends on	lint
Runs in parallel with	Build job

#	Step	Command / Action	Purpose
1	Checkout	<code>actions/checkout@v4</code>	—
2	Setup Node	<code>actions/setup-node@v4</code> (v22)	—
3	Install deps	<code>npm ci</code>	—
4	Prisma generate	<code>npm run db:generate</code>	Generate Prisma client from schema (no DB needed)
5	Test + Coverage	<code>npm run test:coverage</code>	Run all 66 unit tests; enforce 80% coverage gate
6	Upload coverage	<code>actions/upload-artifact@v4</code>	Persist coverage/ folder as downloadable artefact (7 days)

Coverage gate: Configured in `package.json` — branches, functions, lines, statements all require $\geq 80\%$. Build fails if any threshold is breached.

Note: Tests use mocked Prisma, SES, and AuditLogger. No live database connection required.

3 Build

Property	Value
Runner	<code>ubuntu-latest</code>
Depends on	lint
Runs in parallel with	Test job

#	Step	Command / Action	Purpose
1	Checkout	<code>actions/checkout@v4</code>	—
2	Setup Node	<code>actions/setup-node@v4</code> (v22)	—
3	Install deps	<code>npm ci</code>	—
4	Prisma generate	<code>npm run db:generate</code>	Required before TypeScript compilation
5	Compile	<code>npm run build</code>	Compile TypeScript → <code>dist/</code>
6	Upload dist	<code>actions/upload-artifact@v4</code>	Persist compiled <code>dist/</code> as artefact (3 days)

4 Migration Check (main push only)

Property	Value
Runner	ubuntu-latest
Depends on	test AND build (both must pass)
Condition	Only on push to main — skipped on PRs
Secrets required	DATABASE_URL

#	Step	Command	Purpose
1	Checkout	actions/checkout@v4	—
2	Setup Node	actions/setup-node@v4	—
3	Install deps	npm ci	—
4	Drift check	prisma migrate diff	Detect any schema drift between migration files and current schema
5	Deploy	prisma migrate deploy	Apply pending migrations to production database (non-destructive)

Safety: `prisma migrate deploy` only applies pending migrations — it never rolls back or drops data. The drift check step warns if `schema.prisma` has changes not yet captured in a migration file.

4. WORKFLOW YAML — .GITHUB/WORKFLOWS/CI.YML

```
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

env:
  NODE_VERSION: "22"

jobs:
  # — Job 1: Lint & Type Check —————
  lint:
    name: Lint & Type Check
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: node-version: ${ env.NODE_VERSION }, cache: npm
      - run: npm ci
      - run: npm run lint
      - run: npx tsc --noEmit

  # — Job 2: Unit Tests & Coverage —————
  test:
    name: Unit Tests & Coverage
    runs-on: ubuntu-latest
    needs: lint
    steps:
```

```

- uses: actions/checkout@v4
- uses: actions/setup-node@v4
  with: node-version: ${{ env.NODE_VERSION }}, cache: npm
- run: npm ci
- run: npm run db:generate
- run: npm run test:coverage
- uses: actions/upload-artifact@v4
  if: always()
  with: name: coverage-report, path: coverage/

# — Job 3: Build —————
build:
  name: Build
  runs-on: ubuntu-latest
  needs: lint
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with: node-version: ${{ env.NODE_VERSION }}, cache: npm
    - run: npm ci
    - run: npm run db:generate
    - run: npm run build
    - uses: actions/upload-artifact@v4
      with: name: dist, path: dist/

# — Job 4: Migrate (main push only) —————
migrate:
  name: Migration Check
  runs-on: ubuntu-latest
  needs: [test, build]
  if: github.ref == 'refs/heads/main' && github.event_name == 'push'
  env:
    DATABASE_URL: ${{ secrets.DATABASE_URL }}
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with: node-version: ${{ env.NODE_VERSION }}, cache: npm
    - run: npm ci
    - run: npx prisma migrate diff ...
    - run: npx prisma migrate deploy

```

5. REQUIRED GITHUB SECRETS

Secret Name	Used By	Value
<code>DATABASE_URL</code>	migrate job	Neon PostgreSQL connection string — <code>postgresql://user:pass@host/db?sslmode=require</code>

Add via: **GitHub Repo** → **Settings** → **Secrets and variables** → **Actions** → **New repository secret**

6. COVERAGE THRESHOLDS

Configured in `package.json` under `jest.coverageThreshold` :

Metric	Threshold	Current
Branches	80%	✓ Passing

Functions	80%	✓ Passing
Lines	80%	✓ Passing
Statements	80%	✓ Passing

7. RECOMMENDED ENHANCEMENTS

Enhancement	Priority	Description
Branch protection rules	High	Require CI to pass before merging PRs to main
Codecov integration	Medium	Add <code>codecov/codecov-action@v4</code> after test step to track coverage trends
Dependency audit	Medium	Add <code>npm audit --audit-level=high</code> step to lint job
Integration tests	Medium	Add a separate <code>integration</code> job using a PostgreSQL service container
Docker build	Low	Add job to build and push Docker image to GHCR on main push
Release tagging	Low	Add semantic-release or manual tag trigger to publish versioned releases